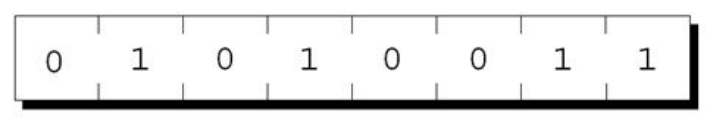
Lesson: Pointers

CSE 4107 : Structured Programming I

Shahriar Ivan

Pointer Variables

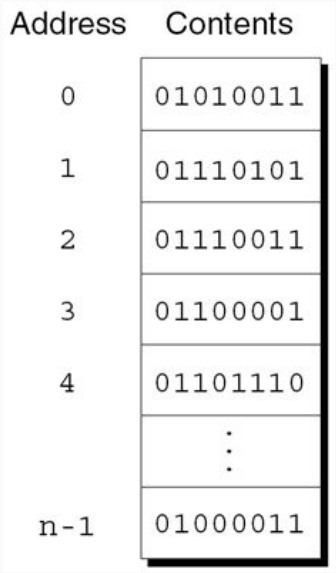
- The first step in understanding pointers is visualizing what they represent at the machine level
- In most modern computers, main memory is divided into ***bytes,*** with each byte capable of storing eight bits of information:



- Each byte has a unique ***address***

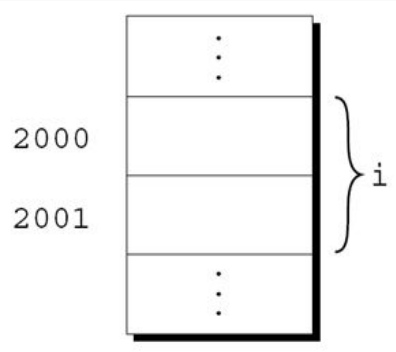
Pointer Variables

- If there are *n* bytes in memory, we can think of addresses as numbers that range from 0 to *n* − 1:



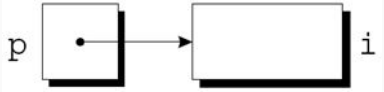
Pointer Variables

- Each variable in a program occupies one or more bytes of memory
- The address of the first byte is said to be the address of the variable
- In the following figure, the address of the variable **i** is *2000*:



Pointer Variables

- Addresses can be stored in special ***pointer variables***
- When we store the address of a variable i in the pointer variable p, we say that p “*points to*” i
- A graphical representation:



Pointer Variables

Declaration

- General form:

  type-name **variable-name*;

  ```
  int *p;
  ```

  Pointer *p* points to *objects* of type int

- Can appear in declaration with along with other variables

  ```
  int i, j , a[10], b[20], *p, *q;
  ```

Pointer Variables

Declaration

- Can point to *objects* of a particular type

```
int *p;          // points only to integers
double *q;       // points only to doubles
char *r;         // points only to characters
```

- Two new operators
  - & (Address) -> ***Finds*** the address of a variable
  - * (Indirection) -> To ***access*** the object a pointer points to

The Address and Indirection Operators

The Address Operator

- Declaration sets aside space for a pointer, doesn't make it point to an *object*
- Crucial to initialize before using

```
int i, *p;
...
p = &i;
```

- Can initialize a *pointer* variable when declared, also it can be combined

```
int i;
int *p = &i;
int j, *q = &j;
```

The Address and Indirection Operators

The Indirection Operator

- If *p* points to *i*, print the value of *i*

```
printf("%d", *p);
```

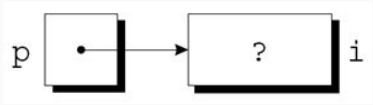
- Can combine two operators

```
j = *&i;    // same as j=i
```

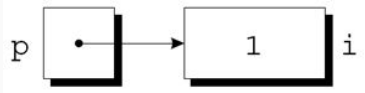
- When *p* points to *i*, **p* is an *alias* for *i*
  - **p* has the same value as *i*
  - Changing **p* changes the value of *i*

The Address and Indirection Operators

```
p = &i;
```

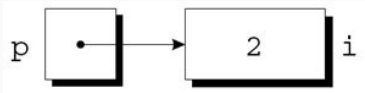



```
i = 1;
```




```
printf("%d\n", i);
printf("%d\n", *p);
*p = 2;
```

/* prints 1 */




```
printf("%d\n", i);
printf("%d\n", *p);
```

/* prints 2 */

The Address and Indirection Operators

The Indirection Operator

- Applying the indirection operator to an uninitialized pointer variable causes undefined behavior:

```
int *p;
printf("%d", *p);          /*** WRONG ***/
```

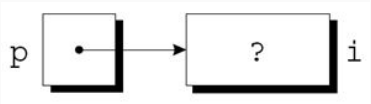
- Assigning a value to *p is particularly dangerous:

```
int *p;
*p = 1;                    /*** WRONG ***/
```

Pointer Assignment

- C allows the use of the assignment operator to copy pointers of same type
- Assume that the following declaration is in effect:

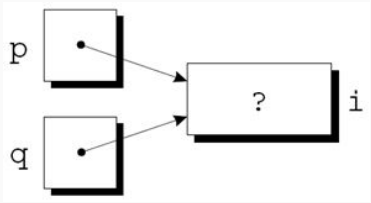
```
int i, j, *p, *q;
p = &i;
```



- Another example of pointer assignment:

```
q = p;
```

q now points to the same place as p:

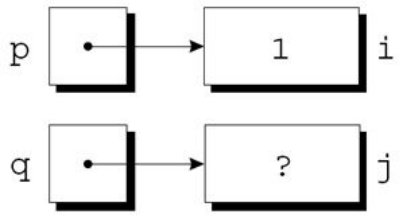


- Can change i using either *p or *q

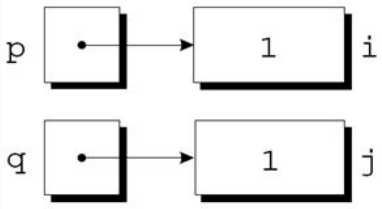
Pointer Assignment

- Do not confuse q = p; (Pointer Assignment) with *q = *p;

```
p = &i;
q = &j;
i = 1;
```




```
*q = *p;
```



Pointers as Arguments

- Can be used to modify arguments:

```
void decompose(double x, long *int_part, double *frac_part){
        *int_part = (long) x;
        *frac_part = x - *int_part;
}
```

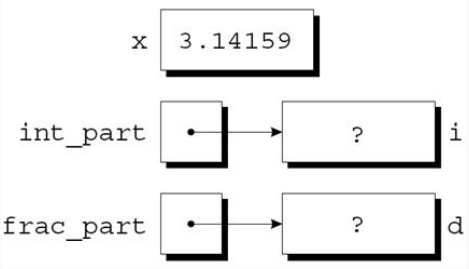
- Possible prototypes:

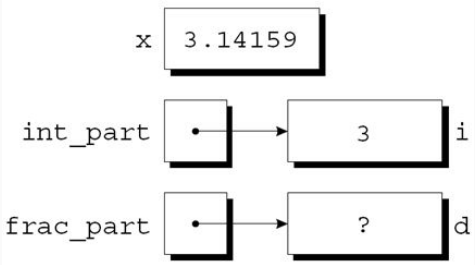
```
void decompose(double x, long *int_part, double *frac_part);
void decompose(double, long *, double *);
```

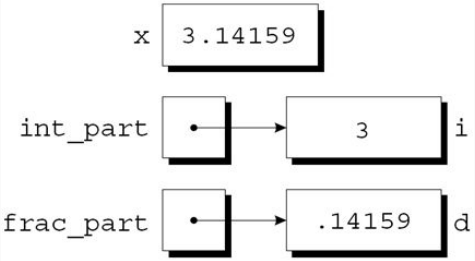
Pointers as Arguments

- A call of decompose:

  decompose(3.14159, &i, &d);







Pointers as Arguments

- Arguments in calls of scanf

```
int i;
scanf("%d", &i);
```

without &, scanf only gets the value of i

- Every argument doesn't need &

```
int i, *p;
p = &i;
scanf("%d", p);
```

- Don't use & with pointers ⇒ `scanf("%d", &p);  /*** WRONG ***/`

Pointers as Arguments

- Failing to pass a pointer to a function when one is expected can have disastrous results
- A call of decompose in which the & operator is missing:

```
decompose(3.14159, i, d);
```

- When decompose stores values in *int_part and *frac_part, it will attempt to change unknown memory locations instead of modifying *i* and *d*
- If we've provided a prototype for decompose, the compiler will detect the error
- In the case of scanf, however, failing to pass pointers may go undetected

Programming Tasks

Finding the Largest and Smallest Elements in an Array

- The *max_min.c* program uses a function named *max_min* to find the largest and smallest elements in an array
- Prototype for *max_min*:

```
void max_min(int a[], int n, int *max, int *min);
```

- Example call of *max_min*:

```
max_min(b, N, &big, &small);
```

Programming Tasks

Finding the Largest and Smallest Elements in an Array

- When *max_min* finds the largest element in *b*, it stores the value in big by assigning it to **max*
- *max_min* stores the smallest element of *b* in small by assigning it to **min*
- *max_min.c* will read 10 numbers into an array, pass it to the max_min function, and print the results:

```
Enter 10 numbers: 34 82 49 102 7 94 23 11 50 31
Largest: 102
Smallest: 7
```

Pointers as Arguments

Using const to Protect Arguments

- General assumption ⇒ Pointers as argument will be modified
- If not ⇒ Use const
- Reason for still passing pointer ⇒ Efficiency
  - Variable might require a large amount of storage
  - Waste of time

```
void f(const int *p){
        *p = 0;              /*** WRONG ***/
}
```

- Compiler can detect such error

Pointers as Return Values

- Functions are allowed to return pointers:

```
int *max(int *a, int *b){
      if (*a > *b)
            return a;
      else
            return b;
}
```

- A call of the *max* function:

```
int *p, i, j;
...
p = max(&i, &j);
```

- After the call, *p* points to either *i* or *j*

Pointers as Return Values

- A function could also return a pointer to an external variable or to a static local variable
- Never return a pointer to an *automatic* local variable:

```
int *f(void){
        int i;
        …
        return &i;
 }
```

- The variable *i* won't exist after *f* returns

Pointers as Return Values

- A function that returns a pointer to the middle element of a, assuming that a has *n* elements:

```
int *find_middle(int a[], int n) {
    return &a[n/2];
}
```